UINTAH · SPICE · gasCombustion

Central-upwind fluxes and a PIMPLE outer loop

Method and implementation notes for the SPICE component

Numerics follow *reactingPimpleCentralFoam* (Kraposhin et al. hybrid KT/PIMPLE solver)

In one paragraph. Detonations computed with ICE + gasCombustion reach the right wave speed and CJ state but show induction zones that are too thin and oscillations right behind the shock. Two causes were targeted. First, ICE builds face states from symmetric, centred averages that carry no upwind dissipation. Second, a single forward-Euler stage is not TVD for limited schemes. This work adds the Kurganov–Tadmor (KT) central-upwind flux construction of reactingPimpleCentralFoam: limited own/nei reconstructions, local wave speeds, α weights, the a_{Sf} dissipation term and the Mach/acoustic-Courant blending function κ. It also adds a PIMPLE-style outer loop that re-runs the whole ICE fluid chain at the latest iterate and combines iterates with a Crank–Nicolson θ rule. KT and the outer loop sit behind input flags that default to off; with them off, SPICE runs ICE's own tasks plus the consistent pressure-work formulation, which SPICE always uses.

Contents

1 Scope and the two switches **2**

2 Where the changes sit in an ICE timestep **2**

3 Limited own/nei face reconstruction **2**

4 Central-upwind weights and the blending function **4**
4.1 Local wave speeds and the α weights 4
4.2 The hybrid blending function κ 4

5 Split mass fluxes **5**

6 Transport: the CentralUpwind advector **5**

7 Face velocity, face pressure and the pressure update **6**
7.1 Face velocity (`faceVelocity`) 6
7.2 Face pressure (`facePressure`) 6
7.3 Pressure update (`ktPressureFlux`) 6

8 The PIMPLE outer loop **7**
8.1 The update rule 7
8.2 How it is realised in Uintah 7
8.3 Why explicit ICE is enough for detonations 8

9 Fidelity map **9**

10 Coupling to MPM: FSPICE **10**
10.1 The FSPICE component . . . 10
10.2 KT next to solids . . . 10
10.3 The frozen-solid outer loop . . . 10
10.4 No-penetration transport (`interfaceTransport donorAcceptor`) . . . 11
10.5 Heat exchange and pressure work next to solids (`interfaceExchange particles`) . . . 12
10.6 Conduction across the interface (`interfaceExchange conduction`) . . . 12
10.7 Tangential momentum at the interface (`interfaceShear`) . . . 13
10.8 Thermal expansion in the Lagrangian specific volume . . . 13
10.9 Defaults and limitations . . . 13

11 Input reference **14**

12 Suggested checks **14**

13 Implementation lessons **15**

File index **16**

1 Scope and the two switches

The work touches explicit ICE with a single ICE material on a single level, reacting through the `gasCombustion` model. Two independent groups of options control it:

Switch	Effect
`<advection type="CentralUpwind">`	Turns on the KT machinery: weights, fluxes, the new advector, the KT face velocity and face pressure, and the KT term in the pressure update (Sections 3–7).
`<PIMPLE> nOuterCorrectors > 1`	Turns on the outer loop (Section 8). It works with any advection type but was designed to be paired with `CentralUpwind`.

The design decisions agreed before implementation were:

- **The GitHub solver is the specification for the numerics.** Earlier Uintah experiments were consulted only for framework plumbing (sub-scheduler data-warehouse handling, ghost widths).
- **Explicit ICE, not impICE.** pimpleCentralFoam is implicit for all-Mach efficiency, not for detonation stability. At the tutorial's `maxCo 0.1`, $\kappa = 1$ at the front and the scheme behaves like explicit KT (Section 8.3).
- **Energy stays in ICE's internal-energy form** with the existing `newPressWork 1` pressure work and kinetic-energy correction. No total-energy rewrite was made.

2 Where the changes sit in an ICE timestep

An explicit ICE step is a Lagrangian phase (pressure, face velocities, momentum and energy sources) followed by an advection (remap) phase and a conversion back to primitive variables. Figure 1 shows the task sequence with the new tasks marked, and the outer loop wrapped around it.

> **In the code** `SPICE/SPICE.cc`: `SPICE::scheduleTimeAdvance` calls `SPICE::scheduleFluidChain(sched, level, inOuterLoop)`, which holds the task span from the thermo properties through `advectAndAdvanceInTime`. The flag `inOuterLoop` replaces `scheduleComputeModelSources` by `schedulePimpleCopyModelSources`.

3 Limited own/nei face reconstruction

Every KT quantity starts from two reconstructions of a cell-centred field at a face: one from the low side (*own*) and one from the high side (*nei*). This matches OpenFOAM's `fvc::interpolate(q, own|nei, "reconstruct(q)")`. On a structured mesh the face f normal to direction d sits between cell P (low) and cell $N = P + \hat{e}_d$ (high):

$$q_{\text{own}} = q_P + \tfrac{1}{2}\,\psi(r_P)\,(q_N - q_P), \qquad r_P = \frac{q_P - q_{P-1}}{q_N - q_P}, \tag{1}$$

$$q_{\text{nei}} = q_N - \tfrac{1}{2}\,\psi(r_N)\,(q_N - q_P), \qquad r_N = \frac{q_{N+1} - q_N}{q_N - q_P}. \tag{2}$$

With a central cell gradient on a uniform mesh, OpenFOAM's gradient-based ratio $r = 2\,(\mathbf{d}\cdot\nabla q)_P/(q_N - q_P) - 1$ reduces to exactly r_P above, so the two constructions agree. The limiter $\psi(r)$ is zero for $r \le 0$ (a local extremum falls back to first order), and otherwise:

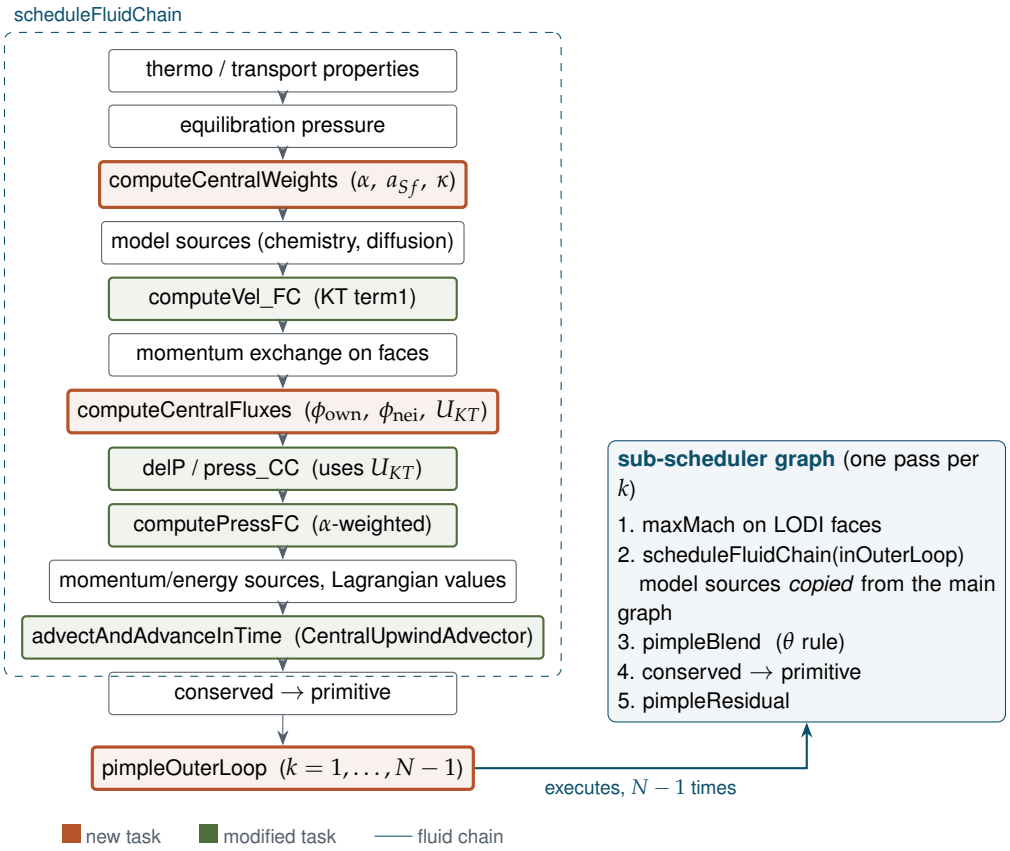


Figure 1: Explicit ICE timestep with the new and modified tasks. The main-graph pass is outer iteration $k = 0$. The model sources are evaluated once, from u^n, in the main graph only.

`limiter=`	$\psi(r)$, $r > 0$	note
`vanAlbada` (default)	$\dfrac{r(r+1)}{r^2+1}$	OpenFOAM definition, used by the reference tutorials
`vanLeer`	$\dfrac{2r}{1+r}$	
`minmod`	$\min(r, 1)$	most dissipative second-order choice
`MC`	$\min\left(2r, \frac{1+r}{2}, 2\right)$	least dissipative
`upwind`	0	first order everywhere

Vector fields are limited component by component. If $|q_N - q_P|$ is at round-off level, then $q_{own} = q_P$ and $q_{nei} = q_N$.

Boundaries. The four-cell stencil needs $P-1$ and $N+1$. Whether they exist is decided from the *level's* cell index range, including extra cells (`Level::findCellIndexRange`) and periodicity, never from the extent of a fetched array. When $P-1$ (or $N+1$) falls outside, that side is taken to first order ($\psi = 0$). The index substituted for the missing cell is P (or N), so no array is ever indexed out of range: C++ evaluates the arguments before the call even when their values go unused.

> **In the code** `SPICE/Core/CentralUpwind.h` (pure math, no Patch or DataWarehouse): `limiter()`, `ownNei()` for scalars and `Vector`, `upwindValue()`, `ktStencilFlags()`.

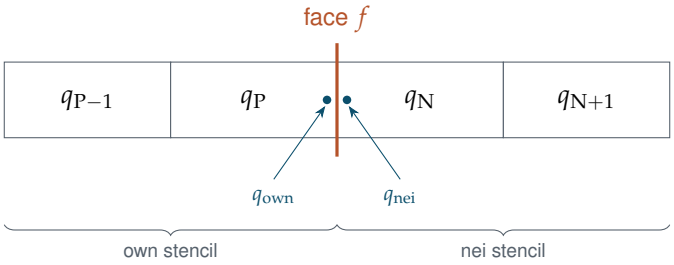


Figure 2: Face convention. In Uintah the face index equals the index of cell N.

> `SPICE/SPICE.cc`: the `Pm1Safe/Np1Safe` idiom at every call site.

4 Central-upwind weights and the blending function

4.1 Local wave speeds and the α weights

With u_{own}, u_{nei} the reconstructed face-normal velocities and c_{own}, c_{nei} the reconstructed sound speeds, the one-sided local speeds of propagation are

$$a^+ = \max\big(u_{own} + c_{own},\ u_{nei} + c_{nei},\ 0\big), \qquad a^- = \min\big(u_{own} - c_{own},\ u_{nei} - c_{nei},\ 0\big). \tag{3}$$

`fluxScheme`	α_{own}	a_{Sf}
`Kurganov` (default)	$\dfrac{a^+}{a^+ - a^-}$	$a^-\,\alpha_{own} \le 0$
`Tadmor`	$\frac{1}{2}$	$-\frac{1}{2}\max(a^+, -a^-)$

with $\alpha_{nei} = 1 - \alpha_{own}$. With these, the KT numerical flux of a conserved quantity U with physical flux F is

$$\mathcal{F}_f = \alpha_{own} F_{own} + \alpha_{nei} F_{nei} + a_{Sf}\,(U_{nei} - U_{own}). \tag{4}$$

Because $a_{Sf} \le 0$, the last term is a dissipation proportional to the jump between the two reconstructions. In smooth flow the two reconstructions nearly coincide and the term vanishes; at a shock the limiter opens the jump and the term switches on. Supersonic flow gives $a^- = 0$, so $\alpha_{own} = 1$ and $a_{Sf} = 0$: the flux becomes pure upwind, from the own side.

4.2 The hybrid blending function κ

reactingPimpleCentralFoam blends between the compressible KT flux and a pressure-based, upwind-limited flux with

$$\kappa = \min\left(\frac{\mathrm{Ma}_f}{\mathrm{Co}_{ac}},\ 1\right), \qquad \mathrm{Ma}_f = \frac{|\alpha_{own} u_{own} + \alpha_{nei} u_{nei}|}{\alpha_{own} c_{own} + \alpha_{nei} c_{nei}}, \qquad \mathrm{Co}_{ac} = \frac{\max(a^+, -a^-)\,\Delta t}{\Delta x_d} \tag{5}$$

(`kappaFunction MachToAcCourant`, the reference's *standardMachToAcCourantRatio*). The value `kappaFunction one` forces $\kappa = 1$. Near a detonation front $\mathrm{Ma} \sim 1$ and $\mathrm{Co}_{ac} \lesssim 0.5$, so $\kappa = 1$ (full KT). In a low-Mach deflagration $\kappa \ll 1$ and the scheme relaxes toward an upwind-limited flux close to baseline ICE.

> **Departure from the reference** The reference computes κ at the end of an outer iteration from the mass fluxes of that iteration, and uses it in the next. Here κ is computed from the reconstructed cell states at the start of each fluid-chain pass. Inside the outer loop this is equivalent to a lag of one iteration.

> **In the code** Task `SPICE::computeCentralWeights`, scheduled right after the equilibration pressure. Inputs: `rho_CC`, `speedSound_CC` (NewDW) and `vel_CC` (OldDW), all at ghost 2, plus `delT`. Outputs: `ktAlphaOwn_FC{X,Y,Z}`, `ktaSf_FC{X,Y,Z}` and `ktKappa_FC{X,Y,Z}`.

5 Split mass fluxes

Once ICE has produced the final face velocity u_f for the step (`uvel_FCME` after the face momentum exchange), the reference's split fluxes are formed:

$$\tilde{u}_{\text{own}} = \alpha_{\text{own}} u_f - a_{Sf}, \qquad \tilde{u}_{\text{nei}} = \alpha_{\text{nei}} u_f + a_{Sf}, \tag{6}$$

$$\rho^{\kappa}_{\text{own}} = \kappa\rho_{\text{own}} + (1-\kappa)\rho_{up}, \qquad \rho^{\kappa}_{\text{nei}} = \kappa\rho_{\text{nei}} + (1-\kappa)\rho_{up}, \tag{7}$$

$$\phi_{\text{own}} = \tilde{u}_{\text{own}}\,\rho^{\kappa}_{\text{own}}, \qquad \phi_{\text{nei}} = \tilde{u}_{\text{nei}}\,\rho^{\kappa}_{\text{nei}}, \tag{8}$$

where ρ_{up} is the own or nei density chosen by the sign of u_f, and $\tilde{u}_{\text{own}}$, $\tilde{u}_{\text{nei}}$ are the reference's `aphiv_own` and `aphiv_nei`. The κ blend of the densities is the reference's blend of `psi_own/nei` toward a single face value. Last comes the reference's `updateKappa` redistribution:

$$\phi_{\text{own}} \leftarrow \phi_{\text{own}} + (1-\kappa)\,\phi_{\text{nei}}, \qquad \phi_{\text{nei}} \leftarrow \kappa\,\phi_{\text{nei}}. \tag{9}$$

Two identities worth knowing. The redistribution leaves the total mass flux $\phi_{\text{own}} + \phi_{\text{nei}}$ unchanged. Writing $\rho_F = \kappa(\alpha_{\text{own}}\rho_{\text{own}} + \alpha_{\text{nei}}\rho_{\text{nei}}) + (1-\kappa)\rho_{up}$,

$$\phi_{\text{own}} + \phi_{\text{nei}} = \rho_F\,u_f + \kappa\,a_{Sf}\,(\rho_{\text{nei}} - \rho_{\text{own}}), \qquad \text{so} \qquad U_{KT} \equiv \frac{\phi_{\text{own}} + \phi_{\text{nei}}}{\rho_F} = u_f + \kappa\,a_{Sf}\,\frac{\rho_{\text{nei}} - \rho_{\text{own}}}{\rho_F}. \tag{10}$$

So the KT face volume flux is ICE's own face velocity plus an acoustic dissipation that is proportional to the density jump and switched by κ. The limits are:

- $\kappa = 1$: the full KT mass flux, $\alpha_{\text{own}}\rho_{\text{own}}u_f + \alpha_{\text{nei}}\rho_{\text{nei}}u_f + a_{Sf}(\rho_{\text{nei}} - \rho_{\text{own}})$.
- $\kappa = 0$: $\phi_{\text{own}} = \rho_{up}u_f$ and $\phi_{\text{nei}} = 0$, which is TVD upwind transport.

> **In the code** Task `SPICE::computeCentralFluxes`, scheduled after the face momentum exchange and before the pressure update. Outputs: `ktPhiOwn_FC*`, `ktPhiNei_FC*` and `ktUvol_FC*` ($= U_{KT}$).

6 Transport: the CentralUpwind advector

ICE advects per-cell extensive quantities Q_L (mass, momentum, internal energy, the `newPressWork` kinetic-energy carrier K_L, specific-volume × mass, and species mass), and adds the net transfer to Q_L. The new advector implements the reference's `fvmDiv(phi_own, q) + fvmDiv(phi_nei, q)` with vanAlbada-limited interpolation:

$$q = Q_L/m_L, \qquad \mathcal{F}_f = \phi_{\text{own}}\,q^{up}(\phi_{\text{own}}) + \phi_{\text{nei}}\,q^{up}(\phi_{\text{nei}}), \qquad q^{up}(\phi) = \begin{cases} q_{\text{own}} & \phi \geq 0 \\ q_{\text{nei}} & \phi < 0 \end{cases} \tag{11}$$

$$\Delta Q_c = \Delta t \sum_d A_d \left(\mathcal{F}_{f_d^-} - \mathcal{F}_{f_d^+} \right), \qquad Q_c^{adv} = Q_{L,c} + \Delta Q_c. \tag{12}$$

Mass is the case $q \equiv 1$. With $\kappa = 1$, the transfer of ρq reproduces the KT flux (4): the a_{Sf} parts of $\tilde{u}_{\text{own}}, \tilde{u}_{\text{nei}}$ carry $a_{Sf}\big((\rho q)_{\text{nei}} - (\rho q)_{\text{own}}\big)$.

For intensive fields (`vol_frac`, used by the pressure update) the advector uses a volume flux: $\Delta q_c = \frac{\Delta t}{V}\sum_d A_d\big(q_{f^-}u^{\star}_{f^-} - q_{f^+}u^{\star}_{f^+}\big)$, where q_f is the limited upwind value and $u^{\star}$ is whatever face velocity the caller passed in (Section 7).

Bullet-proofing. If the outgoing mass of a cell exceeds its Lagrangian mass, or the outgoing volume exceeds the cell volume, the advector requests a timestep recompute, exactly as the existing advectors do. AMR refluxing is not supported, and the advector throws if it is requested.

> **In the code** `SPICE/Advection/CentralUpwindAdvector.{h,cc}`, created by `SPICE::createAdvector` (handed to ICE through `ICE::setAdvectorFactory`) for type `CentralUpwind` with the `limiter` attribute. `SPICE::advectAndAdvanceInTime` hands it the six ϕ arrays through `setCentralFluxes()`. Face transfers are evaluated on demand per face, never cached in a patch-sized buffer: the face shared with a higher-index neighbour patch is not part of a patch's own face range (see Section 13).

7 Face velocity, face pressure and the pressure update

7.1 Face velocity (`faceVelocity`)

ICE's face velocity is a centred, density-weighted average of the two cell velocities (`term1`), plus a pressure-gradient term (`term2`) and gravity. With the flag on, only the average is replaced, by its KT analogue: the reference's `HbyA` interpolated own/nei and weighted by α,

$$\text{term1} = \alpha_{\text{own}}\, u_{\text{own}} + \alpha_{\text{nei}}\, u_{\text{nei}}. \tag{13}$$

The pressure and gravity terms are unchanged, and the flag-off expression is kept verbatim.

7.2 Face pressure (`facePressure`)

The reference builds the momentum pressure force as $\nabla p \approx \text{div}\big((\alpha_{\text{own}} p_{\text{own}} + \alpha_{\text{nei}} p_{\text{nei}})\,\mathbf{S}_f\big)$. Correspondingly,

$$p_f = \alpha_{\text{own}}\, p_{\text{own}} + \alpha_{\text{nei}}\, p_{\text{nei}}. \tag{14}$$

This is an upwind-biased weighted average *without* a jump term. It is not a Riemann star pressure, which carries an impedance jump $Z_L Z_R (u_L - u_R)/(Z_L + Z_R)$ and adds dissipation a second time on top of the swept-volume advection.

Energy consistency. SPICE always forms the flow work from `press_FC` and cell-centred velocities, built term for term to match the momentum pressure source (the consistent pressure-work formulation; ICE keeps the original `delP_Dilatate` form). The KT p_f therefore feeds momentum and energy consistently, with no further change. The kinetic energy that the KT momentum transport dissipates is returned as heat by advecting total energy $E = I + K_L$ with the same KT advector and recovering $I = E_{adv} - |\mathbf{m}_{adv}|^2/(2\,m_{adv})$.

7.3 Pressure update (`ktPressureFlux`)

In explicit ICE the pressure increment is $\delta p_{\text{dil}} = -\big(\sum_m \Delta t\, \nabla\cdot(\theta_m u_f)\big) / \sum_m \kappa_m$, computed through the advector with the face velocity supplied by the caller. With the flag on, that velocity is U_{KT} from (10) instead of u_f. The dilatation then contains the extra term $\kappa\, a_{Sf}(\rho_{\text{nei}} - \rho_{\text{own}})/\rho_F$. This is the explicit counterpart of the convective part of the reference's pressure equation, $\text{div}(\phi^d_{\text{own}} p) + \text{div}(\phi^d_{\text{nei}} p)$: the KT mass-flux divergence with its acoustic upwinding.

> **Departure from the reference** The reference pressure equation also contains the implicit Laplacian $-\text{lap}(D_{p,\text{own}}, p) - \text{lap}(D_{p,\text{nei}}, p)$ and runs `nCorrectors` PISO sweeps. Explicit ICE has no global pressure solve. The closest analogue is the pressure term already in ICE's face velocity. See Section 8.3 for why this is acceptable for detonations.

8 The PIMPLE outer loop

8.1 The update rule

Let X be any conserved per-cell quantity: mass, momentum, specific volume $\times$ mass, species mass, or total energy. Write $\mathcal{C}(\cdot)$ for one pass of the fluid chain, which maps a starting state to its advected state. The main graph computes $X_0^{adv} = \mathcal{C}(X^n)$, and its conversion to primitives is iterate $X^{(0)}$. For $k = 1, \dots, N-1$ the chain is run again from the previous iterate, and the result is combined with the Crank–Nicolson rule:

$$\boxed{X^{(k)} = X^n + (1-\theta)\big(X_0^{adv} - X^n\big) + \theta\big(\mathcal{C}(X^{(k-1)}) - X^{(k-1)}\big)} \tag{15}$$

Each bracket is Δt times the fluid-chain tendency, evaluated at X^n and at the latest iterate respectively. Every quantity inside the chain is therefore re-evaluated at the latest iterate: the KT weights and κ, the face velocities, the pressure, the fluxes and the face states. This matches the reference, which calls `updateCentralWeights` and `updateKappa` in every outer iteration. If the iteration converges, the fixed point is the θ-scheme; $\theta = \frac{1}{2}$ is the tutorial's `ddt CrankNicolson 1`.

Two outer iterations are Heun's method. With $N = 2$ and $\theta = \frac{1}{2}$, write $\mathcal{C}(X) = X + \Delta t\, \mathcal{L}(X)$. Then $X^{(0)} = X^n + \Delta t \mathcal{L}(X^n)$ and

$$X^{(1)} = X^n + \tfrac{\Delta t}{2}\, \mathcal{L}(X^n) + \tfrac{\Delta t}{2}\, \mathcal{L}\big(X^{(0)}\big), \tag{16}$$

which is the two-stage strong-stability-preserving Runge–Kutta method. Unlike forward Euler, it keeps the TVD property of the limited spatial operator under a CFL restriction. This is the most direct reason to expect fewer post-shock oscillations. $\theta = 1$ gives a Picard iteration toward backward Euler.

Energy is blended as total energy. Blending momentum and internal energy separately would create or destroy heat through the nonlinearity of $\frac{1}{2}|\mathbf{m}|^2/M$. Instead, each of the four states $s \in \{n,\ adv_0,\ \mathcal{C},\ k-1\}$ is converted to $E_s = I_s + \frac{1}{2}|\mathbf{m}_s|^2/M_s$, E is blended with (15), and the internal energy is recovered as $I^{(k)} = E^{(k)} - \frac{1}{2}|\mathbf{m}^{(k)}|^2/M^{(k)}$. The internal energies at n and $k-1$ are built exactly as the Lagrangian phase builds them: $M\, e_s(T, Y)$ from the model that owns the caloric equation of state (gasCombustion), or Mc_vT otherwise.

Chemistry is evaluated once per step. As in the reference (`reaction->correct()` before `pimple.loop()`), the model sources (chemistry and diffusion) are computed from u^n in the main graph and copied unchanged into every outer iteration. Because both brackets of (15) contain the same frozen source, the source enters once, with weight one.

Residual. After each outer iteration the code reports $r_k = \max_c \max\big(|\rho^{(k)} - \rho^{(k-1)}|/\rho^{(k)},\ |T^{(k)} - T^{(k-1)}|/T^{(k)}\big)$, a global maximum. With `outerTolerance` > 0 the loop stops early once $r_k <$ `outerTolerance`. The default is a fixed count, as in the reference.

8.2 How it is realised in Uintah

1. **Sub-scheduler.** `d_pimpleSched` is created in `problemSetup` with the data-warehouse mapping ParentOld→0, ParentNew→1, Old→2, New→3 (coarse = fine on a single level). Its graph (Figure 1) is compiled once and recompiled whenever the main graph is.
2. **Driver task.** `pimpleOuterLoop` runs in the main graph right after conserved→primitive, *once per processor* (per-processor patch set, as with impICE's `implicitPressureSolve`), because `execute()` of the sub-graph is collective.

3. **Seeding.** The $k = 0$ primitives (`rho_CC`, `vel_CC`, `temp_CC`, `sp_vol_CC`, `press_CC`, `speedSound_CC`, `specific_heat`, species) are transferred from ParentNewDW into a primed sub-DW. Each iteration then rotates the sub-DWs so that the previous iterate becomes the sub-OldDW. `delT`, `timeStep` and `simTime` are put back in every iteration.
4. **Captured advected state.** Right after the main-graph advection, a small task copies `mass_adv`, `mom_adv`, `eng_adv`, `sp_vol_adv` and every species `var_adv` into `*_adv0` labels. `pimpleBlend` reads X_0^{adv} from them, which avoids requiring a label from ParentNewDW while modifying the same label in the sub-NewDW.
5. **Copy-back.** After the last iteration the primitives (including `mach`, `press_CC` and species) are copied into ParentNewDW over *extra cells too*, so the next step starts from the final iterate's boundary values as well as its interior values.
6. **Timestep control.** A recompute or abort request raised inside the sub-graph is forwarded to the main graph.

> **In the code** `SPICE/SPICE.cc` (each with its `schedule...` pair):
> `pimpleOuterLoop`, `capturePimpleAdv0`, `pimpleCopyModelSources`, `pimpleBlend`, `pimpleResidual`.
> `SPICE/Core/SPICELabel.*`: `mass_adv0`, `mom_adv0`, `eng_adv0`, `sp_vol_adv0`, `pimpleResidual`.
> Per-iteration residual print: `SCI_DEBUG="SPICE_PIMPLE:+"`.

8.3 Why explicit ICE is enough for detonations

pimpleCentralFoam is pressure-implicit so that it can take acoustic Courant numbers $\gg 1$ at low Mach, where $\kappa \to 0$ and the scheme becomes pressure-based. In the detonation tutorials, `maxCo 0.1` keeps $\mathrm{Co}_{ac} \approx 0.1$–$0.3$ at the front, so $\kappa = 1$ and the implicit Laplacian is a small correction. A large implicit step would push κ toward zero and smear the shock and the induction zone. The stability that matters here comes from the KT dissipation and the TVD time integration, both of which the explicit version has. An impICE variant would pay off only for runs that also need long, low-Mach phases, such as deflagration-to-detonation transition.

9 Fidelity map

reactingPimpleCentralFoam	**Uintah SPICE (this work)**
`reconstruct(rho, U, p, psi)`, own/nei, vanAlbada	`centralUpwind::ownNei`, default limiter vanAlbada
`ap`, `am`, `alpha_own`, `aSf` (Kurganov / Tadmor)	`computeCentralWeights`
`kappa` (standardMachToAcCourantRatio)	`computeCentralWeights`; κ-blend and redistribution in `computeCentralFluxes`
`aphiv_own/nei`, `phi_own/nei`, `updateKappa`	`computeCentralFluxes`
`fvmDiv(phi_own,·)+fvmDiv(phi_nei,·)` for U, Y, h	`CentralUpwindAdvector` (mass, momentum, $E = I + K_L$, specific volume, species)
`gradp` with α-weighted p_{own}, p_{nei}	`computePressFC` (`facePressure`)
`HbyA` interpolated own/nei with α	`computeVelFace` term1 (`faceVelocity`)
pEqn convective terms $\text{div}(\phi^d p)$	U_{KT} in the explicit pressure update (`ktPressureFlux`)
pEqn Laplacian, `nCorrectors`	not implemented (would need impICE)
`reaction->correct()` once per step	model sources once per step, frozen in the outer iterations
`pimple.loop()`, `nOuterCorrectors`	`pimpleOuterLoop` sub-scheduler
`ddt CrankNicolson 1`	θ rule (15), $\theta = 0.5$
`hEqn` (sensible enthalpy, KChange, KSource)	ICE internal energy with SPICE's consistent pressure work; total-energy blend in the loop
`maxCo 0.1`	`<cfl>` in the input (0.1 in the detonation input)

Departure from the references, collected

- κ is lagged by one outer iteration relative to the reference (Section 4).
- Transport inside one outer iteration is explicit, where OpenFOAM assembles implicit `fvm` matrices. The outer iterations supply the implicit coupling.
- There is no implicit pressure Laplacian and no PISO `nCorrectors`.
- Diffusion is frozen with chemistry across outer iterations. The reference re-solves diffusion inside `YEqn`/`hEqn`.
- Energy is carried as internal energy, not sensible enthalpy.

10 Coupling to MPM: FSPICE

10.1 The FSPICE component

FSPICE (`<SimulationComponent type="fspice"/>`, or `"rfspice"` with rigid MPM) schedules ICE, MPM and SPICE tasks the way MPMICE schedules ICE and MPM. It owns a SPICE object, which owns the ICE object, and an MPM object, and it carries its own copies of the MPMICE tasks: the MPMICE of the PIMPLE/KT development merged with the public MPMICE changes since (MPM reduction subsets, `scheduleConservedQtys`). MPMICE itself is neither used nor modified. The input is the `<CFD><SPICE>` block of this note plus an `<MPM>` block. With no MPM materials FSPICE runs exactly as SPICE.

The gas-solid interface models live in SPICE's tasks (Table 1); without MPM materials every one of them is inert, and the stand-alone SPICE component refuses MPM materials. FSPICE adds the tasks that need the particles: the sharp solid fraction `computeSolidVolFracP`, the frozen-solid capture and copy tasks of the outer loop, and its copy of MPMICE's equilibration pressure.

Task	With MPM materials
`computeCentralWeights`	open-face weight `ktOpen_FC*`; $\kappa \to w\kappa$
`computeVel_FC`, `computePressFC`	KT face velocity / pressure blended to the stock forms by w
`computeCentralFluxes`	donor–acceptor transport on faces with $w < 1$, `gasFaceFrac_FC*`
`computeDelPressAndUpdatePressCC`	MPM `vol_frac` with a SecondOrder advector; complementary face partition
`accumulateEnergySourceSinks`	mixture pressure work over all matls; placeholder gas; interface conduction and shear heat
`accumulateMomentumSourceSinks`	interface shear
`computeLagrangianSpecificVolume`	thermal-expansion term scaled by TMV (Section 10.8)
`advectAndAdvanceInTime`	donor values on no-penetration faces
`pimpleOuterLoop`	gas-only chain, solid cells restart from time n
exchange (`SPICE/Exchange`)	particle-aware heat exchange, tangential slip

Table 1: SPICE tasks that carry the FSPICE gas-solid models.

10.2 KT next to solids

A cell whose gas volume fraction is below $1 - s$ is a solid cell (s = `solidCellFrac`, default 10^{-3}). Each face gets an open weight

$$w = \mathrm{clamp}\left(\frac{\min(f_L, f_R) - (1 - 10s)}{9s},\, 0,\, 1\right), \tag{17}$$

which is 1 between open-gas cells and 0 on any face touching trace gas. κ is multiplied by w, which turns the face into the stock upwind flux with the ICE face velocity, and the KT face velocity and face pressure are blended with the stock density-weighted forms by the same w. Without this, KT's a_{Sf} term moves trace gas in and out of the solid at the sound speed. In the pressure equation the MPM materials keep the stock SecondOrder advector; only the gas uses the KT advector. With the KT advector, gas stayed trapped inside a moving piston.

10.3 The frozen-solid outer loop

MPM runs once per step, in the $k = 0$ pass. The outer iterations re-run the gas tasks only (`FSPICE::scheduleGasChainForPimple`, handed to SPICE through its PIMPLE hooks). The solid inputs are copies of the $k = 0$ pass: cell-centred velocity, temperature and specific volume captured right after `interpolateNCToCC_0`, plus density and Lagrangian mass, momentum and energy. Solid cells restart from time n in every outer iteration, so they get $(1 - \theta)X^0 + \theta X_{\mathrm{sub}}$: conservative and positive for any θ, and first order in time as in stock MPMICE. The residual

skips them. Their pressure restarts from time n too: in a solid cell the equilibration pressure is essentially the one it starts from, so starting from the $k = 0$ result added the solid's dilatation pressure change once per outer iteration (a deforming wall's cell-pressure swing doubled at $N = 2$ and collapsed on unloading). If the $k = 0$ pass asks for a recompute or abort the outer loop is skipped, with all ranks agreeing. The outer loop requires the scalar exchange model without convective heat transfer. Known imbalance: the solid keeps the $k = 0$ exchange while the gas gets the θ-blended one.

10.4 No-penetration transport (`interfaceTransport donorAcceptor`)

Why. Gas leaks into MPM solids even with the 10^{15} drag when gas and solid share one face velocity: a rigid wall's first cell fills to 0.3 gas. Every scheme, stock SecondOrder too, carries the *donor cell's average composition* through a face; the driver is either a small face velocity into the wall from the gas-to-wall-cell pressure jump, or wall deformation. A drag or slip exchange model cannot stop this, since there is no relative velocity to remove: the no-penetration condition has to decide *which material crosses a face*. KT made it worse: vanAlbada is far less compressive at a steep front than SecondOrder's Barth–Jespersen gradient, and the gas (KT) and MPM (SecondOrder) face fractions in the pressure equation did not sum to one.

The solid fraction. `FSPICE::computeSolidVolFracP` locates the solid with a sharp field. The nodal solid volume fraction $r_n = 2^f \sum_m g\text{Volume}_m / V_{\text{cell}}$ (f the number of non-periodic domain faces the node lies on) is 1 inside the solid at any compression, $\frac{1}{2}$ at a face-aligned surface and 0 in the gas. A cell's solid fraction is the part of it where the trilinear interpolant of r exceeds $\frac{1}{2}$. EqPress's `vol_frac` is not used: it inherits the node-averaged cell mass and smears a sharp surface over about two cells. Particle boxes (the first version) were dropped: MPM's deformation gradient drifts from the particle spacing and opened holes inside a stretched wall.

What it does. On faces with $w < 1$, `SPICE::computeCentralFluxes` applies the Hirt–Nichols (1981) donor–acceptor rule with that solid fraction:

1. Geometry: the acceptor's or the donor's solid fraction s_X, chosen by the dominant component of ∇s at the donor, gives the gas volume $V_g = (1 - s_X)V_f + \max(s_X V_f - s_D V, 0)$. Gas enters a solid acceptor only once the donor's solid is used up; trace gas in a solid donor leaves toward the gas first; between two gas cells this is exactly upwind.
2. Acceptor: a cell its particles cover ($s \geq 0.9$) takes no gas; a trace-gas cell the surface is receding from takes gas only from a particle-free donor and only into the volume its particles have left; a cell with a real gas pocket stays compressible.
3. No trace gas moves between two trace-gas cells inside the solid (all 8 nodes carry MPM mass).
4. At most half a donor's gas leaves per step, never below the material's `tiny_rho`.
5. Momentum, energy, species and `sp_vol` on these faces use the donor's own specific values (first order).

Each test is a linear ramp, not a switch: with switches a round-off difference opened a cell one step early and a 3D periodic detonation left its 1D twin by 4×10^{-2} in gas fraction; with the ramps the two agree to round-off. The gas share of each face, `gasFaceFrac_FC*`, also sets the face partition in the pressure equation: gas V_g / V_f, each MPM material its part of the rest.

Placeholder trace gas. Trace gas in a cell inside the solid (all 8 nodes carry MPM mass) is a placeholder in `FSPICE::computeEquilibrationPressure`: it takes no part in the Newton iteration, its micro density stays at its start-of-step value, and it deforms with its host. When a solid in tension drove its cell pressure to the 10^{-12} clamp, the ideal-gas volume inflated and its f_θ share of the solid's pressure work grew to 10^4 times its own energy per step.

T2, KT	before	donorAcceptor
gas 2–3 cells into the wall	0.17 / 0.10	10^{-12}
rigid wall, first wall cell	0.33	10^{-12}
wall stretched to $\lambda = 1.5$	crash at 28 µs	runs 1.4 ms
T3, H_2 autoignition: gas inside the wall	2.5×10^{-2}	10^{-12}

10.5 Heat exchange and pressure work next to solids (`interfaceExchange particles`)

Why. The gas-side cell of a wall holds about 20% node-averaged "solid" and no particles, and the stock 10^{15} exchange still acts there: the gas is held at the wall temperature (the near-wall density overshoot, which KT spreads over about four cells) and at the wall's speed (the reflected shock forms one cell early).

What it does.

1. No gas–solid heat exchange in a particle-free cell (`solidVolFracP_CC` below 0.1 and not inside the solid), ramped over 0.05–0.15. Cell-centred momentum exchange stays stock: removing the drag there let the smear's velocity crush the wall's front particle layer.
2. No gas–solid face momentum exchange between two particle-free cells: there the lock held the gas at the smear's face velocity and gas could not refill a cell the wall cooled.
3. Pressure work in the conservative face form for all gas, none for placeholder trace gas, whose host's cell pressure is $\sim 150\times$ its own.
4. EqPress in cells without solid: the gas pressure from its mass over its volume, as in pure ICE. Stock MPMICE normalizes the material volumes by their total, so even in a cell holding only gas the pressure followed the advected specific volume, not the mass.

Metal walls need the time step to resolve the solid's sound speed: MPMICE's CFL counts only the gas, and steel or aluminium walls next to 1500 K gas stopped within 1 µs at `timestep_multiplier` 1.0 and ran at 0.5.

T2, KT	stock	donorAcceptor	+ particles
heat 0: gas T next to the wall (exact 492 K)	—	824 K	473 K
heat 0: reflected shock / interface T vs exact	—	+9.3% / −5.3%	+0.9% / −0.2%
10^{15}: reflected shock / interface p vs exact	+2.6% / −2.1%	+2.6% / −2.2%	+0.9% / −0.8%

10.6 Conduction across the interface (`interfaceExchange conduction`)

Everything `particles` does, plus: no gas–solid heat exchange in any cell (the `<heat>` coefficient is not used for gas–MPM pairs) and heat conducted across the interface located by `solidVolFracP_CC` instead (`SPICE::interfaceConduction`), between the gas and solid temperature nodes nearest it, with k_g = `thermalCond_CC` and k_s the MPM `thermal_conductivity`. It conducts only when the gas material's `<thermal_conductivity>` is nonzero. Each side delivers the flux its near-interface profile implies: the one-sided second-order gradient when the thermal layer spans the first cell, and otherwise the heat-balance integral of an erfc layer,

$$\delta = \frac{\pi}{2}\, w\, \frac{T_2 - T_1}{T_2 - T_i}, \qquad q = k\, \frac{T_2 - T_i}{\delta}, \tag{18}$$

exact for a semi-infinite body and without stored history. The interface temperature T_i makes the two fluxes equal. Each pair is point implicit, $\Delta E = G\,\Delta t\,\Delta T / (1 + \Delta t \sum G_g/C_g + \Delta t \sum G_s/C_s)$, and the gas loses exactly what the solid gains. Inputs are start-of-step values, so every outer iteration books the same energy, and the result does not depend on the patch layout. Two semi-infinite bodies (gas 330 K, wall 300 K): heat into the wall 1.025 / 1.024 / 1.022 of exact at 0.2 / 0.5 / 1 ms, wall surface 307.27 K (exact 307.52 K).

10.7 Tangential momentum at the interface (`interfaceShear`)

With `stock`, the 10^{15} cell-centred exchange holds the gas in front of a solid at the solid's velocity in every direction: a no-slip wall half a cell off the surface at any viscosity. `slip` keeps that exchange only along the surface normal $\mathbf{n} = \nabla s/|\nabla s|$ in cells without particles, so the gas slides freely. `model` adds the wall shear from the near-wall velocity profile (`SPICE::interfaceShear`): the conduction layer model with interface value 0 (no slip at the surface), $k \to \mu$ and $\alpha \to \nu$, point implicit; the kinetic energy it dissipates is heat in the gas cell. Air viscosity at 100 m/s over steel: momentum to the wall +1 to +3% and shear +3 to +5% of Stokes' solution, against +85% and -65 to -28% with `stock`. `slip` and `model` are for walls aligned with the grid: where the surface is a staircase, gas sliding along it crosses into locked cut cells and the face-form pressure work books compression their gas does not undergo.

10.8 Thermal expansion in the Lagrangian specific volume

In `computeLagrangianSpecificVolume` the thermal-expansion term is a fraction of the material's own volume $m\,v = f\,\mathrm{TMV}$ (MPMICE normalizes `vol_frac` by the total material volume; pure ICE has $\mathrm{TMV} = V$). With V in its place the change was scaled by V/TMV, and gas cooled by a wall in a cell with $\mathrm{TMV} < V$ lost more than its whole volume. With TMV an ideal gas gives $v_L/v_{\mathrm{old}} = T_{\mathrm{new}}/T_{\mathrm{old}}$, always positive. The compression term keeps V.

10.9 Defaults and limitations

With KT on and MPM materials present the interface options default to `donorAcceptor`, `conduction` and `model`; KT and PIMPLE themselves stay opt-in. FSPICE requires one level and exactly one ICE material; `rfspice` uses rigid MPM. The interface options need the scalar exchange model, and the outer loop with MPM materials needs it without convective heat transfer.

11 Input reference

Tag	Default	Meaning
`<CFD><SPICE><advection type="CentralUpwind" limiter="...">`		
`limiter`	`vanAlbada`	vanAlbada, vanLeer, minmod, MC, upwind
`<CFD><SPICE><CentralUpwind>` (optional; requires the advection type above)		
`fluxScheme`	`Kurganov`	or `Tadmor`
`kappaFunction`	`MachToAcCourant`	or `one` ($\kappa \equiv 1$)
`faceVelocity`	`true`	KT term1 in the face velocity
`facePressure`	`true`	α-weighted face pressure
`ktPressureFlux`	`true`	U_{KT} in the pressure update
`solidCellFrac`	`1e-3`	FSPICE: gas fraction below $1 - s$ is a solid cell; $0 < s < 0.1$
`interfaceTransport`	see text	FSPICE: `upwind` or `donorAcceptor`
`interfaceExchange`	see text	FSPICE: `stock`, `particles` or `conduction`
`interfaceShear`	see text	FSPICE: `stock`, `slip` or `model`
`<CFD><SPICE><PIMPLE>` (optional)		
`nOuterCorrectors`	`1`	total passes per step; 1 = loop off; the reference uses 2
`theta`	`0.5`	θ in (15); $0 < \theta \leq 1$
`outerTolerance`	`0`	0 = fixed count; otherwise early exit on r_k
`nCorrectors`	`1`	parsed, reserved for an impICE variant

Enforced at start-up. SPICE runs on a single level (no AMR) with exactly one ICE material and no MPM materials: like reactingPimpleCentralFoam it is a single-fluid code, with species carried as transported scalars. MPM materials need FSPICE (Section 10), whose interface options default to `donorAcceptor`, `conduction` and `model` when KT is on and MPM materials exist, and to `upwind`, `stock` and `stock` otherwise. The outer loop requires explicit ICE and no turbulence model. CentralUpwind requires explicit ICE.

Provided inputs. In `StandAlone/inputs/SPICE/`:

- `riemann_sm.ups`, `riemann_sm_KT.ups`, `riemann_sm_PIMPLE.ups`, `riemann_sm_KT_PIMPLE.ups`: the Sod shock tube of `inputs/ICE/riemann_sm.ups` with each combination of KT advection and the outer loop ($N = 2$, $\theta = 0.5$).
- `riemann_gc.ups`: the same shock tube with gasCombustion supplying the thermodynamic data (`idealAir.xml`, no chemistry), KT and PIMPLE. It exercises the outer loop's transported-scalar and model-source paths.

12 Suggested checks

1. **Oscillations.** Compare `nOuterCorrectors` 1 and 2 with everything else fixed. This isolates the effect of the time integration from that of the flux.
2. **Induction zone.** If it shortens or collapses, first set `facePressure false`. A dissipative face pressure applied as a cell-centred source is the known way to over-dissipate ICE's momentum equation. Then compare `ktPressureFlux true/false`.
3. **Blending.** Compare `kappaFunction one` with `MachToAcCourant`. The detonation front should already sit at $\kappa = 1$; inspect `ktKappa_FCX`.
4. **Limiter.** Compare minmod, vanAlbada and MC to bracket limiter sensitivity.

5. **Time rule.** Compare $\theta = 0.5$ with $\theta = 1$, and use `nOuterCorrectors 3` to check the convergence of the residual r_k with `SCI_DEBUG="ICE_PIMPLE:+"`.
6. **Regression.** Remove all new tags. The original code paths should reproduce the pre-change results.

13 Implementation lessons

Five defects surfaced during testing. Each is a general Uintah pitfall and is recorded here so it is not rediscovered.

Out-of-range stencil arguments

Passing `q[Pm1]` to a function that ignores it when the stencil is invalid still reads that memory, and at a domain edge with one extra cell this segfaulted. The fix is to substitute P or N before indexing (`Pm1Safe`/`Np1Safe`).

Patch-sized face buffers

`Patch::getSFC*HighIndex()` includes the upper face only when the patch has no neighbour there. A face buffer filled over a patch's own face range therefore lacks the face $c + 1$ of the patch's last cell at an interior patch boundary. Reading it gave negative specific volume at every patch seam of the 12-rank flame run. Face transfers are now evaluated on demand from ghosted data.

Sub-scheduler driver scheduled per patch

A task that calls a sub-scheduler's collective `execute()` must be added on the per-processor patch set. On `level->eachPatch()` it ran once per patch, each time with only one patch seeded. This failed as soon as a rank owned more than one patch (4 ranks, 12 patches), and went unnoticed with one patch per rank.

Copy-back over interior cells only

The outer loop's final state has to be copied over extra cells too. Otherwise the next step's boundary values come from iteration $k = 0$, which matters for LODI and Neumann boundaries.

Limiter overflow (found in 2D)

The textbook vanAlbada and vanLeer forms, $r(r+1)/(r^2+1)$ and $2r/(1+r)$, evaluate $\infty/\infty = \mathrm{NaN}$ when r is huge. That happens when the jump across a face is tiny next to a finite upwind jump, which is routine for trace species on an oblique 2D front. A 2D channel detonation crashed with NaN temperatures after four steps. For $r > 1$ the code now uses the equivalent forms $(1 + 1/r)/(1 + 1/r^2)$ and $2/(1 + 1/r)$.

File index

`ICE/Core/CentralUpwind.h`	limiters, own/nei reconstruction, KT weights, κ (new)
`ICE/Advection/CentralUpwindAdvector.{h,cc}`	KT transport (new)
`ICE/Advection/Advector.h`	basket extended with the ϕ arrays
`ICE/Advection/AdvectionFactory.cc`, `sub.mk`	registration, `limiter` attribute
`ICE/ICE.{h,cc}`	flags, `scheduleFluidChain`, KT tasks, face velocity and pressure, pressure update, outer loop
`ICE/Core/ICELabel.{h,cc}`	`kt*_FC*`, `*_adv0`, `pimpleResidual` labels
`StandAlone/inputs/UPS_SPEC/ice_spec.xml`	new tags
`ICE/PIMPLE_KT.md`	short usage note
`SPICE/Exchange/InterfaceScalarExch.{h,cc}`	FSPICE: particle-aware exchange, tangential slip
`SPICE/Advection/InterfaceSecondOrderAdvector.{h,cc}`	FSPICE: MPM `vol_frac` with the face partition
`FSPICE/FSPICE.{h,cc}`	FSPICE: SPICE + MPM, the MPMICE task copies, solid fraction, frozen-solid outer loop